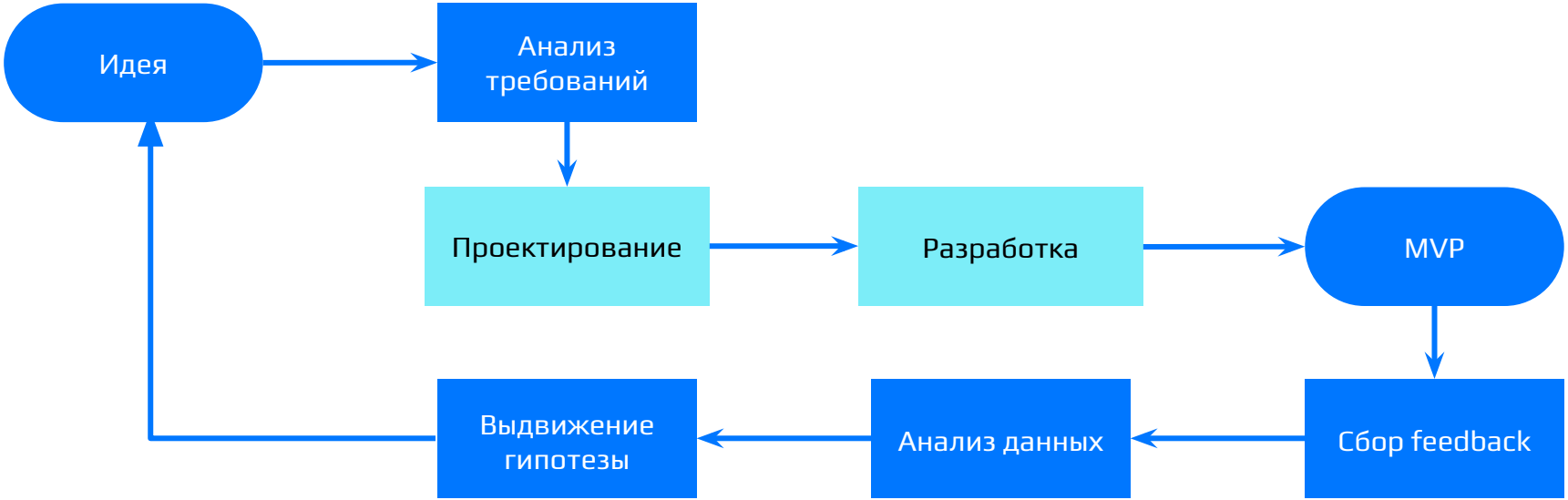
education

Проектирование

Олег Гибадулин

Этапы



education

Проектирование

education

Как не надо

Code-First

Как это выглядит: Бэкендер открывает консоль базы данных и начинает создавать таблицы: CREATE TABLE users..., CREATE TABLE coworkings.... Потом он начинает писать код (контроллеры, сервисы), который просто достает данные из этих таблиц и отдает наружу.

В чем фатальная ошибка: В этом подходе база данных диктует интерфейс. Бэкендер отдает на мобилку то, что ему удобно было положить в базу, а не то, что реально нужно пользователю на экране

Последствия для проекта: Фронты сидят без дела, 5 запросов на 1 экране, негибкость к изменениям.

Hype-Driven Development

Как это выглядит: Разработчик говорит: «Я тут читал, что Netflix использует микросервисы, Kubernetes и брокер сообщений Kafka. Нам срочно нужно делать так же.

В чем фатальная ошибка: Технология выбирается не под задачу бизнеса, а потому что она модная.

Последствия для проекта: Удорожание разработки и поддержки.

Resume-Driven Development

Как это выглядит: Разработчик хочет перейти на следующую работу с повышением зарплаты. Ему нужно вписать в резюме строчку: «Опыт работы с языком Rust и графовыми базами данных».

В чем фатальная ошибка: Архитектура становится заложником личных амбиций одного человека.

Последствия для проекта: Удорожание разработки и поддержки.

Архитектура «Башня из слоновой кости»

Как это выглядит: Архитектор запирается в кабинете на месяц. Он не общается с программистами, не смотрит на реальные навыки команды. Он рисует в Miro гигантскую, идеальную, безупречную схему из 500 квадратиков.

В чем фатальная ошибка: Полный отрыв от реальности. Красивая схема на бумаге часто не учитывает многие факторы.

Последствия для проекта: Такая архитектура никогда не увидит свет.

Без проектирования

Как это выглядит: В коде нет никаких границ (контекстов). Сервис отправки писем напрямую лезет в базу данных платежей, чтобы проверить статус. Модуль скидок напрямую вызывает модуль логистики.

В чем фатальная ошибка: Нарушение принципа слабой связности (Low Coupling). Всё зависит от всего.

Последствия для проекта: Time2Market стремится к бесконечности.

Как надо

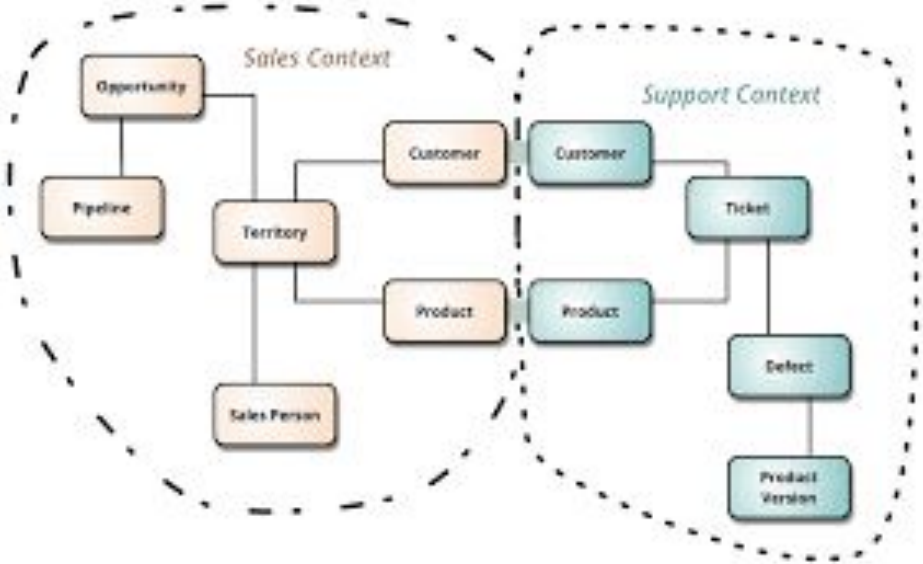
DDD

education

Domain-Driven Design

Решает проблему непонимания между бизнесом и разработкой

Проектирование основывается на бизнесе (доменной области бизнеса)



education

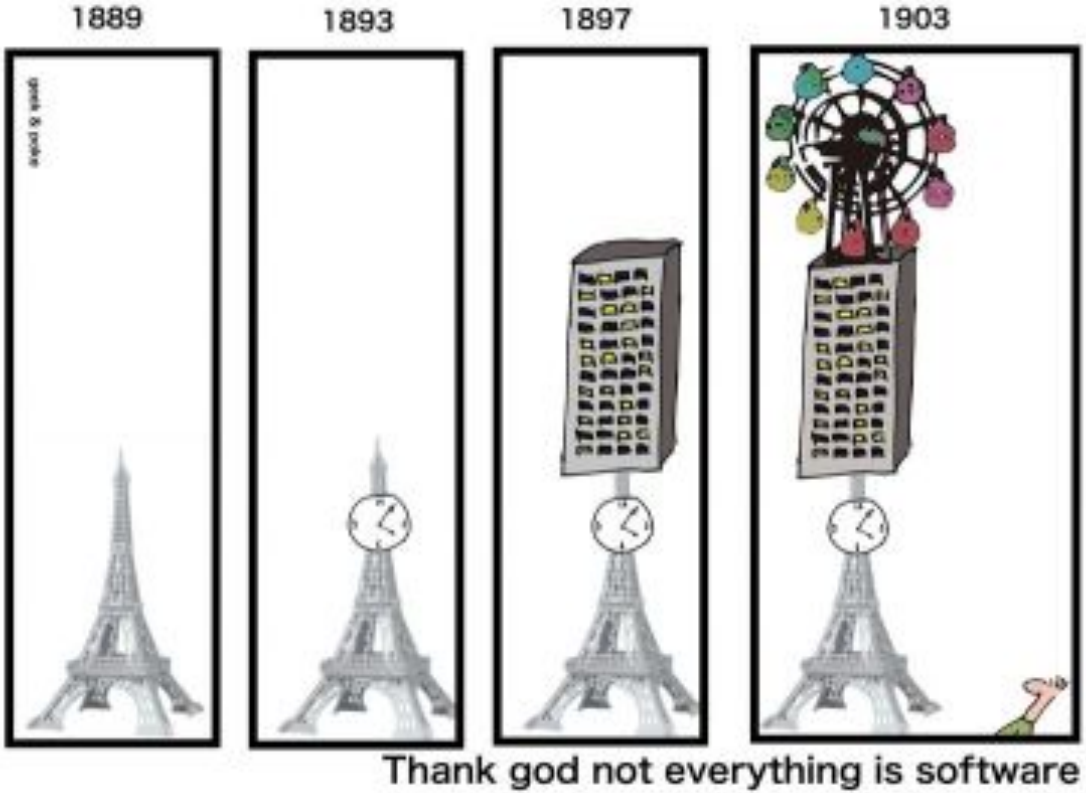
Испорченный телефон

education

Единый язык

- **Проблема**: Бизнес говорит «Подписчик». Аналитик пишет в ТЗ «Клиент». А разработчики в iOS-приложении и на бэкенде называют этот класс User, потому что им так привычнее. Когда происходит баг с «Подписчиком», разработчики полдня ищут, к какому User это относится.
- **Как по DDD**: Команда садится с экспертами из бизнеса и составляет глоссарий. Если договорились использовать слово «Подписчик», то в коде обязательно должен появиться класс Subscriber. В макетах это Subscriber. В базе данных — Subscribers. Никаких синонимов. Код становится документацией.

God Object



education

Разделяй и властвуй

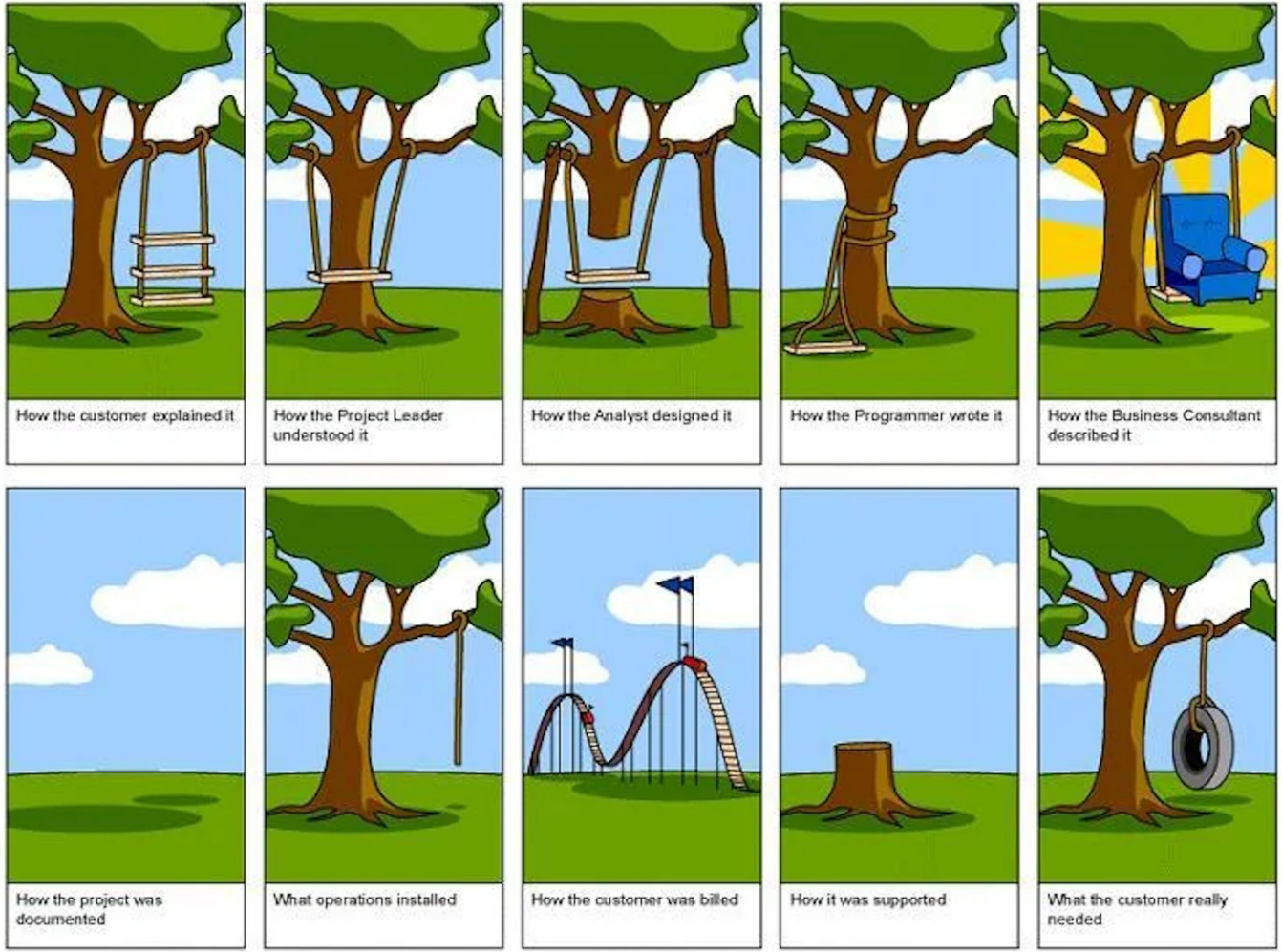
- Мы признаем, что слово «Закладки» значит разные вещи в разных отделах бизнеса.
- Мы делим систему на изолированные зоны.

Архитектура = Структура команд

- DDD — это инструкция по тому, как пилить систему на микросервисы.
- Один ограниченный контекст = зона ответственности одной команды.
- Если границы контекстов определены верно, команды могут релизиться независимо друг от друга.

BDD

education


How the customer explained it
How the Project Leader understood it
How the Analyst designed it
How the Programmer wrote it
How the Business Consultant described it
How the project was documented
What operations installed
How the customer was billed
How it was supported
What the customer really needed

education

Behavior-Driven Development

GIVEN (Контекст) + WHEN (Действие) = THEN (Результат).

- Мы описываем не как система устроена внутри, а что она делает снаружи.
- Используем единый, понятный всем формат.
- Код пишется исключительно для удовлетворения этих сценариев. Ни строчкой больше.

Сценарий 1

Дано: пользователь находится на экране корзины.

И: сумма товаров в корзине составляет 1000 рублей.

И: история заказов этого пользователя пуста (это его первый заказ).

Когда: пользователь вводит промокод NEWBIE200 (скидка 200 руб).

Тогда: промокод применяется успешно.

И: итоговая сумма к оплате меняется на 800 рублей.

И: появляется зеленая галочка "Скидка применена".

Сценарий 2

Дано: пользователь находится на экране корзины.

И: сумма товаров в корзине составляет 1000 рублей.

И: **в истории пользователя уже есть 3 успешно доставленных заказа.**

Когда: пользователь вводит промокод NEWBIE200.

Тогда: итоговая сумма к оплате остается 1000 рублей.

И: появляется красный текст ошибки "Этот промокод действует только на первый заказ".

Требования = Критерии приемки = Тесты

PM пишет текст -> Инструмент (Allure) делает из него тест -> Программист пишет код -> Тест зеленеет.

- Ваша документация никогда не устареет, потому что если код изменится — тест упадет.
- Разработчик точно знает, когда задача готова: когда все сценарии BDD горят зеленым.

Когда применять BDD?

- Идеально для сложной бизнес-логики (оплаты, штрафы, доступы).
- Идеально для UI-сценариев (пользовательские пути в iOS/Android).
- Не нужно применять для простых CRUD-операций (создать профиль, удалить профиль).

API-First

education

Противоположность Code-First

Вместо того, чтобы отталкиваться от реализации бека и быть заблоченым им, клиент и бек садятся вместе и договариваются о контратках.

Магия параллелизма

- Команды независимы друг от друга.
- Если бэкенд задерживается, фронт всё равно готов на 100% (протестирован на моках).
- Если нужно поменять дизайн, мобилка не ждет бэкенд — она просит обновить контракт.

Чек-лист менеджера на старте фичи

- Макеты готовы?
- Фронт и бэкенд обсудили макеты?
- Swagger-контракт написан и утвержден обеими сторонами?
- Только теперь разрешаем переводить задачи в статус In Progress.

EDA

education

Event-Driven Architecture

В EDA центром вселенной становится Событие (Event) — железобетонный факт того, что что-то уже произошло в прошлом.

Сервисы больше не вызывают друг друга напрямую. Они ничего друг о друге не знают. Они просто кричат в общую шину данных (Брокер сообщений): "Эй, статус заказа изменился!". А те, кому это интересно, слушают шину и реагируют.

Подходы к проектированию

- API-First распараллеливает работу людей
- DDD наводит порядок в бизнес-логике
- BDD спасает от сломанного ТЗ
- EDA спасает систему от обрушения под нагрузкой

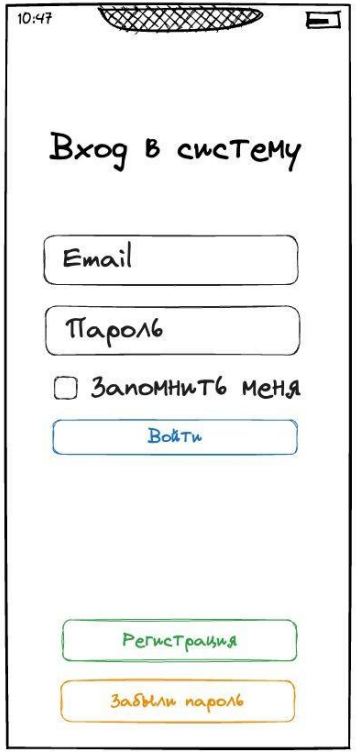
Вопросы?

education

Практика

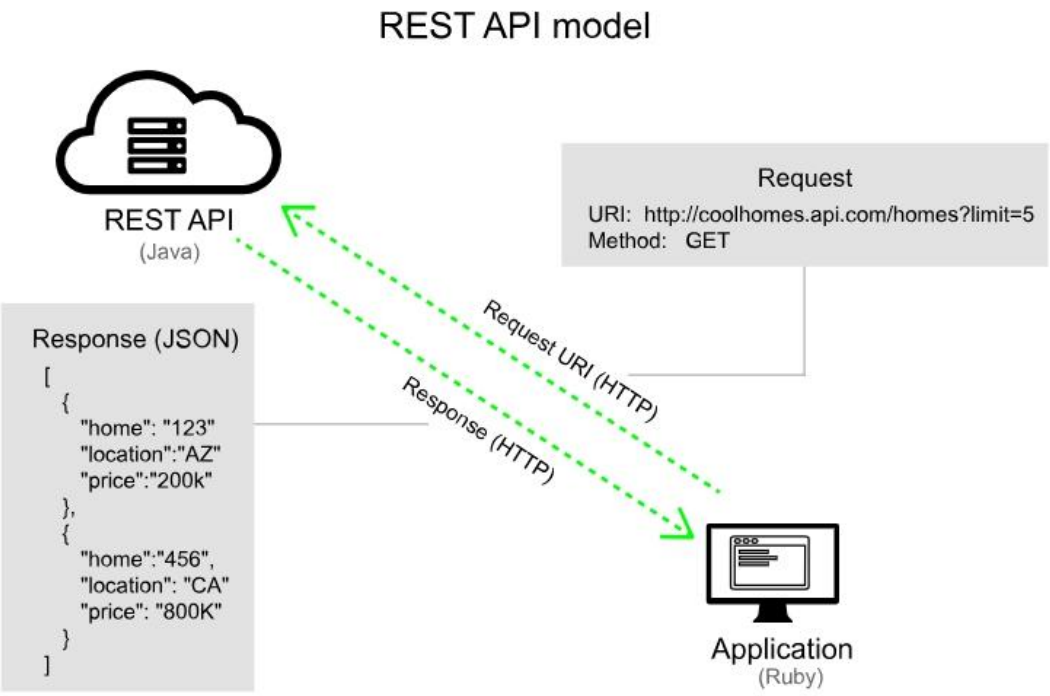
education

Фрейминг



education

Rest API



education

ER-диаграмма

education

Практическое задание №3

- **DDD**: Разделите всё ваше приложение на 4-5 доменных зон с краткими описаниями и глоссариями.
- **BDD**: Выберите 1 Критический путь MVP. Опишите его в формате Дано-Когда-Тогда: 1 сценарий успеха и 1 сценарий отказа.
- **Wireframes**: Набросайте квадратиками 2-4 ключевых экрана, по которым проходит пользователь в ваших сценариях из критического пути.
- **API-First**: Спроектируйте JSON-контракты (запрос и ответ) для 1–2 главных ручек.

Спасибо за внимание!

Не забудьте оставить отзыв :)

education